

The C++ programming language

December 10, 2025
Ramasamy Kandasamy

Contents

1	Language	2
1.1	Pre-processor	2
1.2	Expressions	2
1.3	Declarations	3
1.4	Initialization	3
1.5	Functions	4
1.6	Statements	4
1.7	Classes	5
1.8	Templates	6
1.9	Miscellaneous	7
2	Utils	7
2.1	Data Elements	7
2.2	Memory management in C++	7
2.3	limits	7
3	Iterators	7
4	Containers	7
4.1	Sequential Containers	7
4.2	Associative Containers	8
4.3	Container Adapters	8
5	Algorithms	8
6	Libraries	9
6.1	Input output	9
6.2	cin and cout	10
6.3	Miscellaneous	11
6.4	String	11
7	Devtools	11
7.1	G++	11
7.2	GDB	11

1. Language

1.1. Pre-processor

File Inclusion:

- `#include <filename>` — Search system include paths.
- `#include "filename"` — Search current directory first, then system paths.

Macro Definitions:

- `#define NAME replacement`
- `#define FUNC(arg1, arg2) expression`
Eg: `#define MAX(A,B) ((A) > (B) ? (A) : (B))`
- `#undef NAME` — Remove definition of NAME.

Macro Operators:

- `#` (stringification) — Converts argument to string literal.
Eg: `#define STR(x) #x` ⇒ `STR(hello)` expands to `"hello"`
- `##` (token pasting) — Concatenates tokens.
Eg: `#define CONCAT(a,b) a##b` ⇒ `CONCAT(x,y)` expands to `xy`

Conditional Compilation:

- `#if constant-expression / #elif / #else / #endif`
- `#ifdef NAME` — True if NAME is defined.
- `#ifndef NAME` — True if NAME is not defined.
- `defined(NAME)` — Can be used in `#if` expressions.

Example header guard:

```
#ifndef HEADER_FILE_H
#define HEADER_FILE_H
// ... declarations ...
#endif
```

- `#pragma once` — Modern single-inclusion guard (non-standard but widely supported).

Diagnostics:

- `#error "message"` — Emit compile-time error.
- `#warning "message"` — Emit warning (compiler extension).
- `#line N "file"` — Change reported line number and filename in diagnostics.

Predefined Macros:

<code>__FILE__</code>	Current source file name (string).
<code>__LINE__</code>	Current line number (integer).
<code>__DATE__</code>	Compilation date as "Mmm dd yyyy".
<code>__TIME__</code>	Compilation time as "hh:mm:ss".
<code>__cplusplus</code>	C++ standard version (e.g., 201703L for C++17).
<code>__func__</code>	Current function name (C++11, not technically a macro).
<code>__STDC_HOSTED__</code>	1 if hosted implementation, 0 if freestanding.

1.2. Expressions

Value Categories

<code>lvalue</code>	Has name. Can take address.
<code>prvalue</code>	Temporary value. No name. <i>Guaranteed copy elision.</i>
<code>xvalue</code>	Expiring. Eg: <code>std::move(x)</code> .
<code>glvalue</code>	<code>lvalue</code> $\cup$ <code>xvalue</code> .
<code>rvalue</code>	<code>prvalue</code> $\cup$ <code>xvalue</code> .
<code>L-value reference</code>	Refers to an <code>lvalue</code> . Eg: <code>int& ref = var;</code>
<code>R-value reference</code>	Refers to an <code>rvalue</code> . Eg: <code>int&& rref = 5;</code>

Exceptions:

- R-value can bind to `const` l-value reference.
Eg: `const int& x = 42;`
- String literals are l-values.

Literals

Boolean	<code>true</code> <code>false</code>	
Integer	<code>42</code> <code>4'200'000</code> <code>0b101010</code> <code>052</code> <code>0x2a</code>	Decimal. Decimal with separators. Binary. Octal. Hexadecimal.
Float	<code>3.14</code> <code>3.14f</code> <code>6.626e-34</code>	Double. Float. $x \rightarrow x \times 10^y$
Character	<code>'a'</code> <code>'\n'</code> <code>'\13'</code> <code>u8'a'</code>	Ordinary literal. Using escape seq. UTF-8.
String	<code>"Hello"</code>	String literal.
Pointer	<code>nullptr</code>	

NOTE: Literals are prvalues.

Exception: String literals are lvalues.

`std::initializer_list`

Constructed automatically from braced-init-list.

Eg:

```
Foo foo({1, 2, 3});
print_list({1, 2, 3});
```

Passed as argument to functions.

Eg:

```
void print_list(std::initializer_list<int> list);
Foo(std::initializer_list<int> list);
```

Access elements using iterators or range-based for loop.

Arithmetic and logical operation

	Operators	Associativity
1	<code>() [] -> .</code>	left to right
2	<code>! ~ ++ -- + - * & (type) sizeof</code>	right to left
3	<code>* / %(remainder)</code>	left to right
4	<code>+ -</code>	left to right
5	<code><< >></code>	left to right
6	<code>< <= > >=</code>	left to right
7	<code>== !=</code>	left to right
8	<code>&</code>	left to right
9	<code>^</code>	left to right
10	<code> </code>	left to right
11	<code>&&</code>	left to right
12	<code> </code>	left to right
13	<code>?:</code>	left to right
14	<code>= += -= *= /= %= &= ^= = <<= >>=</code>	right to left
15	<code>,</code>	left to right

Selected operators

`>>` Bitwise right shift operator.
`a = b >> 1` equivalent to dividing `b` by 2.

`<<` Bitwise left shift operator.
`a = b << 1` equivalent to multiplying `b` by 2.

`?:` Ternary conditional operator.
`x = (condition)? expr1: expr2;`
`expr1` → `condition == true`.
`expr2` → `condition == false`.

Type casting

<code>const_cast</code>	Modifies <code>const</code> /volatile qualifiers. Modifying a <code>const</code> object leads to UB.
<code>static_cast</code>	Compile-time downcast. Not safe.
<code>dynamic_cast</code>	Safe downcast. Eg: <code>Base* b = static_cast<Base*>d;</code> when invalid <code>b</code> is <code>nullptr</code> . Eg: <code>Base& b = static_cast<Base&>d;</code> when invalid throws <code>std::bad_cast</code> .
<code>reinterpret_cast</code>	

1.3. Declarations

<delc-type-specifier> <declarator>

Examples:

```
int x; // <int> <x>
int *x; // <int> <*x>
const int *x; // <const int> <*x>
int *const x; // <int> <const *x>
const int *const x; // <const int> <const *x>
```

Declarator

Tells the compiler about the name of the variable, its type (pointer, array, function etc).
Could also have cv qualifiers (const, volatile).
Eg: `int *const p;`
Here `const` is part of the declarator.

decl-type-specifiers

Fundamental types	
char	
int	
float	
double	
void	
Common aliases	
size_t	
char_t	
int8_t	
int64_t	
uint32_t	
NOTE: <code>std::size_t</code> , <code>int8_t</code> are preferred over <code>size_t</code> , <code>int8_t</code> etc. Why?	
register	Ignored by modern compilers.
mutable	Removes const-ness from a const object member variable.
inline	Relaxes ODR. NOTE: Global linkage.
static	Specifies internal linkage. <i>Local:</i> initialized only once, retains value between calls. <i>Global:</i> visible only in the current TU. Different from inline. <i>See "Class" for static members.</i>
extern	Specifies external linkage. Eg: <code>extern int x; // x is now available in other TUs.</code>
thread_local	Specifies thread local storage duration. Eg: <code>thread_local int x;</code>

typedef	typedef <type> <alias>; Eg: typedef long long int ll;
using	using <alias> = <type>; Eg: using ll = long long int; using supports template aliase, but typedef does not.
namespace	Declares a namespace. Eg: namespace Foo { ... } Alias: namespace Bar = Foo;
auto	Type deduction based on initializer. Eg: auto x = 10; // x is int
decltype	Deduces type of an expression. Eg: decltype(x) y; // y has same type as x decltype is exact, unlike auto. Eg: auto doesn't preserve references, const, arrays decay to pointers etc.
static_assert	Compile time assertion. static_assert(<bool-constexpr>) static_assert(<bool-constexpr>, " <message> ")

enum, enum class, union

enum Eg:

```
enum Color { RED,          // 0
             GREEN = 5,    // 5
             BLUE          // 6
};
```

- No scope:Eg: `Color c = RED;`.
- Implicitly converts to int. Eg: `int x = RED;`.

enum class (scoped enum) Eg:

```
enum class Color {
    Red,          // 0
    Green = 3,    // 3
    Blue          // 4
};
```

Eg usage: `Color c = Color::Red;`
Union
Eg:

```
union Data {
    int i;
    float f;
};
```

1.4. Initialization

Fundamental Types

```
int a;          // Garbage value.
int a{};        // a = 0.
int a{1};       // a = 1;
int a = b;      // Copy intialization.
```

Array types

```
int a[3];          // Garbage value.
int a[3]{};        // a = {0, 0, 0}
int a[3]{1};       // a = {1, 0, 0}
int a[3]{1, 2, 3}; // a = {1, 2, 3}
```

Aggregate Types

Aggregates: Arrays, "Simple" structs/classes without Ctors.
Consider:

```
class Foo {
public:
    int x;
    double y;
};

Foo foo;
Foo foo{};          // foo.x = 0; foo.y = 0.0;
Foo foo{1};         // foo.x = 1; foo.y = 0.0;
Foo foo{1, 2};      // foo.x = 1; foo.y = 2.0;
```

Class Types with Constructors

```
Foo foo;          // Default Ctor.
Foo foo();        // Default Ctor.
Foo foo{};        // Default Ctor.
Foo foo{bar};     // Copy Ctor. Not blocked by explicitly.
Foo foo(bar);     // Copy Ctor. Not blocked by explicitly.
Foo foo = bar;    // Copy Ctor. Blocked if explicit.
Foo foo(1, 2);    // Parameterized Ctor.
Foo foo{1};       // Ctor with std::initializer_list.
Foo foo{1, 2};    // Ctor with std::initializer_list.
Foo foo({1, 2});  // Ctor with std::initializer_list.
```

1.5. Functions

- **Definition :** `T foo(args) {...}`.
- **Declaration :** `T foo(args);`.
- **Variadic Arguments:** `T foo(int n, ...)`.
C-style. Not safe. Use variadic templates.

The main() function

```
main(int argc, char *argv[]) {  
    ...  
}  
argc      No. of args including the program name.  
argv[0]    Pointer to executable name.  
argv[i]    Pointer to ith argument.
```

Lambda function

Usage: [`<capture>`](`<params>`)->`<return-type>`{`<body>`}

`<capture>` Specifics

`[]` No variables captured.

`[=]` All variables captured by value.

`[&]` All variables captured by reference.

`[x, &y]` x captured by value, y by reference.

`[=, &y]` All variables except y by value, y by reference.

NOTE: Not specifying return type is not compile time error and ofter compiler can deduce it. But, it is good practice to specify it, especially for complex lambdas.

Eg:

```
// Sort descending using lambda  
sort(nums.begin(), nums.end(), [](int a, int b) -> bool {  
    return a > b;  
});
```

Passing functions as arguments

1. Function pointers.
Eg:

```
int foo(int a, int b, int (*bar)(int, int)) {  
    return bar(a, b);  
}  
// Function call.  
foo(5, 3, add);
```

Doesn't work with lambdas with captures.

2. `std::function`

```
int foo(int a, int b, std::function<int(int, int)> bar) {  
    return bar(a, b);  
}  
// Function call.  
foo(5, 3, add);
```

3. Templates

```
template <typename bar>  
int foo(int a, int b, bar bar) {  
    return bar(a, b);  
}  
// Function call.  
foo(5, 3, add);
```

Zero overhead

1.6. Statements

If-else	<pre>if (<i>expression</i>) <i>statement</i> else if (<i>expression</i>) <i>statement</i> : : else <i>statement</i></pre>
Switch	<pre>switch (<i>expression</i>) { case <i>const-expr</i>: <i>statements</i>; break; case <i>const-expr</i>: <i>statements</i>; break; default: <i>statements</i>; break; } Without break statement the execution will <i>fall through</i> all the cases. This can be used as: switch (<i>expression</i>) { case 1: case 2: case 3: group = 0; break; case 4: case 5: group = 1; break; default: group = -1; break; }</pre>
While	<pre>while (<i>expression</i>) <i>statement</i></pre>
For	<pre>for (expr1; expr2; expr3) <i>statement</i> This for loop is equivalent to: expr1; while (expr2) { <i>statements</i>; expr3; }</pre>
Do While	<pre>do <i>statement</i> while (<i>expression</i>);</pre>
Break	break : Exit the the loop or control. Works for while, for, do while and switch

Continue	continue: Continue next iteration. Works for while, for, do while If switch is within a loop and if continue is used inside switch and if it is executed, then the loop skips to next loop.
Goto and label	Example: if (disaster) { goto error; } ... error: { statement }

Shorter alternative for if-else statement:
x = <cond-expr> ? <out-if-true> : <out-if-false>;
Eg: char is_pos = (n > 0) ? 'y' : 'n';

1.7. Classes

Eg:

```
class Point {
public:
    Point(int x, int y) : x_(x), y_(y) {}
private:
    int x_;
    int y_;
};
```

Inheritance Behaviour

Simple rule

1. Private members are never inherited.
2. derived-type = min(base-type, inheritance-type).

Member / Inheritance	Public	Protected	Private
Public	Public	Protected	Private
Protected	Protected	Protected	Private
Private	NA	NA	NA

Members Variables

- this - pointer to the current object.
- static - shared across all objects of the class.

Member Functions

Rule-of-5 members

1. Copy Ctor

```
// Pass by ref to avoid recursion.
Vec::Vec(const Vec &other) {
    n_ = other.n_;
    data_ = new int[n_];
    for (int i = 0; i < n_; i++) {
        data_[i] = other.data_[i];
    }
}
```

2. Move Ctor

```
Vec::Vec(Vec &&other) noexcept {
    n_ = other.n_;
    data_ = other.data_;
    other.n_ = 0;
    other.data_ = nullptr;
}
```

3. Copy Assignment Operator

```
Vec& Vec::operator=(const Vec &other) {
    if (this != &other) { // Self-assignment check.
        delete[] data_; // Free existing resource.
        n_ = other.n_;
        data_ = new int[n_];
        // Deep copy.
        for (int i = 0; i < n_; i++) {
            data_[i] = other.data_[i];
        }
    }
    return *this; // To permit chained assignments.
}
```

4. Move Assignment Operator

```
Vec& Vec::operator=(Vec &&other) noexcept {
    if (this != &other) { // Self-assignment check.
        delete[] data_; // Free existing resource.
        n_ = other.n_;
        data_ = other.data_;
        other.n_ = 0;
        other.data_ = nullptr;
    }
    return *this; // To permit chained assignments.
}
```

5. Destructor

```
Vec::~Vec() {
    delete[] data_;
}
```

Others

6. Default Ctor Eg: Foo::Foo()

7. Converting Ctor

Ctor with single argument or multiple arguments with default values.

```
Say: Foo::Foo(const Bar&);

Foo f = bar; // Implicit conversion from Bar to Foo.

explicit Foo(const Bar&); // Prevents implicit conversion.
```

8. Static methods

Belong to class, not object.
No this pointer.

Compilers provided default implementation for 1-6 methods.

- **delete:** Prevents default generation.
- **default:** Makes the intent clear.
Forces default generation?
- **explicit:** Prevents implicit conversions for converting ctors.

Virtual Methods and Abstract Class

- **Virtual method:** Eg: virtual void Animal::speak();
- **Pure virtual method:** Eg: virtual void Animal::speak() = 0;
- **Abstract class:** Class with at least one pure virtual method. Cannot instantiate objects of abstract class.

Virtual vs normal methods:

```
Animal* a = new Dog();
a->speak();
```

- **speak** is virtual: Dog's speak() called.
- **speak** is normal: Animal's speak() called.

override & final

Specifiers at the end. Relative order between them doesn't matter.

override:

Tells a method is meant to override a base class method.

Compiler error if it doesn't.

Eg:

```
Base: void Animal::speak() const;
Derived: void Dog::speak(); // Compiles.
Unexpected runtime behaviour:
a->speak(); // Base class speak.
Derived w/ override:
void Dog::speak() override; // Compiler error.
```

final:

Method not supposed to be overridden in derived classes.

Compiler error if it is.

Eg:

```
Base: void Dog::speak() override final;
Derived:
void BorderCollie::speak() override; // Compiler error.
```

1.8. Templates

Template Parameter Kinds

Type	typename T or class T Accepts any type as argument.
Non-type	int N, size_t N, auto N, etc. Accepts compile-time constant values. Types: integral, pointers, references, nullptr_t, enums, float (C++20).
Template template	template <typename> class C template <typename> typename C (C++17+) Accepts template as argument.

Template Parameter Syntax

Basic	typename T int N template <typename> class C
With default	typename T = int int N = 10 template <typename> class C = std::vector
Variadic	typename... Args int... Ns template <typename> class... Containers

NOTE: **typename** vs **class** for type parameters are interchangeable. **typename** is preferred (more explicit). Also, **typename** has another use: disambiguating dependent types. Eg: **typename T::value_type x;**

Template Specialization

Full/Explicit Specialization

Provide custom implementation for specific type(s).

Eg:

```
// Primary template
template <typename T>
class MyClass { /*...*/ };

// Full specialization for bool
template <>
class MyClass<bool> { /*...*/ };
```

Partial Specialization

Specialize for a subset of types. **Only for class templates, not functions.**

Eg:

```
// Primary template
template <typename T, typename U>
class Pair { /*...*/ };
```

```
// Partial specialization: both types same
template <typename T>
class Pair<T, T> { /*...*/ };
```

```
// Partial specialization: pointer types
template <typename T, typename U>
class Pair<T*, U*> { /*...*/ };
```

Full specialization	template <> class C<int> {...} All parameters specified. Works for classes and functions.
Partial specialization	template <typename T> class C<T*> {...} Some parameters remain generic. Only for class templates.

Parameter Packs

Syntax

Declaration	typename... Args int... Ns
Expansion	Args... expands to T1, T2, T3, ... func(Args...)... expands to func(T1), func(T2), ... sizeof...(Args) gives pack size.

Example: Compile-time Fibonacci sequence

```
// Fibonacci calculation
template <int N>
struct Fib {
    static constexpr int value =
        Fib<N-1>::value + Fib<N-2>::value;
};

template <>
struct Fib<0> { static constexpr int value = 0; };
template <>
struct Fib<1> { static constexpr int value = 1; };

// Variadic template to hold sequence
template <int... Ns>
struct FibSeq {
    static constexpr int values[] = { Fib<Ns>::value... };
};

// Usage
constexpr int fibs[] = FibSeq<0, 1, 2, 3, 4, 5>::values;
// fibs = {0, 1, 1, 2, 3, 5}
```

Pack expansion Fib<Ns>::value... expands to Fib<0>::value, Fib<1>::value, ..., Fib<5>::value

1.9. Miscellaneous

2. Utils

2.1. Data Elements

Pair

Eg: `pair<int, int> x = 1,2;`
`x.first` Returns 1.
`x.second` Returns 2.

Tuple

Fixed length collection of heterogenous values.

Eg: `tuple<int, string, double> t;`

Operations related to tuple

`get<i>` Return i^{th} value of t. Eg: `get<0>(t);`.
`make_tuple()` Returns tuple out of individual values.
Eg: `make_tuple(5, "Paul", 23.4)`.
`tie()` Unpacking tuple.
Eg:
`tie(n, s, x) = t;`
`tie(n, std::ignore, x) = t;`

Bit set

Definiton

`bitset<size> s;`

Examples:

`bitset<5> s; bitset<5> s(string("1100110"));`

Operations on Bit set:

`a & b` Bitwise AND.
`a | b` Bitwise OR.
`a ^ b` Bitwise EXOR.
`a` Bitwise negation.

2.2. Memory management in C++

- `void* operator new(size)`
Eg: `myClass* p1 = new myClass;`
`myClass* p1 = new(sizeof(myClass));`
- `void delete(void* ptr)`
Eg: `delete p1; delete(p1);`

2.3. limits

Examples:

- `numeric_limits<int>::min()`
- `numeric_limits<int>::max()`
- `numeric_limits<double>::min()`
- `numeric_limits<double>::infinity()`

3. Iterators

4. Containers

4.1. Sequential Containers

Vectors

Definiton:

```
vector<T> v;  
Element access  
operator[]  
v.back()    Last element  
v.front()   First element
```

Modifiers

`v.push_back(5)` Add 5 to the vector. Simillar to push in stack data structure.
`v.insert()` Inserts elements / range in v.
Eg: Append u to v.
`v.insert(v.end(), u.begin(), u.end());`
`v.emplace` `v.emplace{iterator pos, Args}`. New element is constructed in place using `Args` and inserted at pos.
`v.emplace_back` `v.emplace_back{Args}`. New element is constructed in place using `Args` and inserted at the end.
IMPORTANT: Note the differences between pushback, insert, emplace and emplaceback.
`v.pop_back()` Remove element on top of the stack.
`v.erase()` Remove element / range from v.
`v.clear()` Clears all the elements in v.

Capacity

`v.size()` Returns the size of the vector.
`v.capacity()` Current memory allocated to v in terms of number of elements.
`v.max_size()` Maximum memory available for v.
`v.reserve()` Reserve a minimum size for v. Does not affect `v.size()`.
`v.shrink_to_fit()` Shrink the capacity to fit the size.
`v.empty()` Test if v is empty.

`tie` Can be used to unpack a vector.

Example: `tie(a, b, c) = v;`

Queue

Eg: `queue<int> q;`
`q.push(5)` Adds element to the end.
`q.pop()` Removes first element.
`q.front()` Returns first element.
`q.empty()` True if q is empty.

Dequeue

Eg: `deque<int> d;`
`d.push_back(5)` Add to the top.
`d.push_front(2)` Add to the bottom.
`d.pop_back()` Remove from the top.
`d.pop_front()` Remove from the bottom.

4.2. Associative Containers

Set

Definition

`set<type> s;`
`multiset<type> s;`
`unordered_set<type> s;`
`unordered_multiset<type, hasher> s;`
 hasher: Refer Custom Hash Function.

Set operations

`s.insert(item);`
`s.erase(item);`
`s.count(item);`
`s.find(item);` Returns the iterator that points to item.
`s.insert()` Inserts elements in s.
 Eg: Insert elements from t to s.
`v.insert(v.end(), u.begin(), u.end());`

`s.size()`
 Not available for `unordered_set` and `unordered_multiset`.

`s.lower_bound(x)` Returns iterator to the smallest element that is larger than or equal to x.
`s.upper_bound(x)` Returns iterator to the smallest element that is larger than x.

NOTE: erase method remove all instances of the item in the multiset.

Map

`map` : Balanced binary tree.
`unordered_map`: Hash list.

Definition:

`map<type1, type2> m;`
 Eg: `map<string, int> m;`
`unordered_map<type1, type2, hasher> m;`
 hasher: Refer Custom Hash Function.

Operations on map

`m["banana"]` Retrive value associated with **banana**.
`m.count("banana")` Return 1 if the key is present else 0.
`m.size();`

4.3. Container Adapters

Stack

Eg: `stack<int> s;`
`s.push(5)`
`s.top()`
`s.pop()`

Priority queue

Format:

`priority_queue<type, vector<type>, compare>`

Max priority queue [Default]

Eg: `priority_queue<int> q;`

Min priority queue

`priority_queue<int, vector<int>, greater<int>> q;`
`q.push(3)`
`q.top()` Returns the largest element.
`q.pop()` Removes the largest element.

5. Algorithms

Sort and search

Search and sort from C

- `void *bsearch(const void *key, const void *base, size_t n, size_t size, int (*cmp)(const void *key1, const void *datum))`
 Searches `base[0] ... base[n-1]` for `key`.
 Comparison function must return negative if its first argument (search key) is less than its second argument (a table entry) and so on.
- `qsort(void *base, size_t n, size_t size, int (*cmp)(const *void, const *void))`

Search and sort from C++

Compare functions:

```
bool compare(x, y){
    return(x strictly precedes y)
}
```

In all the below, a could be an iterator or a pointer and all of them can use custom compare functions.

- `y = lower_bound(a, a+n, x)`
 Returns an iterator to the first element whose value is $\geq x$.
- `y = upper_bound(a, a+n, x)`
 Returns an iterator to the first element whose value is $> x$.
- `y = equal_range(a, a+n, x)`
 Returns a pair of iterators pointing to the upper bound and lower bound of x.
- `y = equal_range(a, a+n, x, compare.function)`
 Returns a pair of iterators pointing to the upper bound and lower bound of x.
- `sort(it1, it2, compare.function);`

- `stable_sort(it1, it2, compare.function);`
- `is_sorted(it1, it2, compare.function);`
- `is_sorted_until(it1, it2, compare.function);`
 Returns the iterator to the element until which the array is sorted.
- `find_if(start_it, end_it, func)`
 Returns iterator to the first element in the range `[start_it, end_it)` for which `func` returns true.
- `find_if_not(start_it, end_it, func)`
 Returns iterator to the first element in the range `[start_it, end_it)` for which `func` returns false.

Min Max

- `max(a, b, comp); min(a, b, comp);`
- `max({a1, a2, a3}, comp); min({a1, a2, a3}, comp);`
- `max_element(a, a+n, comp);`
 Returns the iterator for the largest element in a. If there are multiple largest element, returns the iterator/pointer for the first one.
- `min_element(a, a+n, comp);`

Merge

- `merge(InputIterator F1, InputIterator L1, InputIterator F2, InputIterator L2, OutputIterator R);`
 Returns the iterator to the end of output.
 NOTE: This operator does not allocate memory. The `OutputIterator R` should point to a vector with sufficient memory to store the concatenated vector.
- `set_intersection` Arguments and output are the same as that for `merge`.
- `set_union` Arguments and output are the same as that for `merge`.
- `set_difference` Arguments and output are the same as that for `merge`.
- `set_symmetric_difference` Arguments and output are the same as that for `merge`.

Hash Function

Can be used for custom types in `unordered_map` and `unordered_set`.

Usage:

```
hash<T> hasher;
size_t hasvalue = hasher(x); //Where x is an object of type T
```

Eg:

```
hash<int> h; size_t x = h(10);
hash<string> h; size_t x = h("hello");
```

Custom hash function.

Custom hash function for a user defined type can be defined as follows:


```

class CustomHash {
public:
    size_t operator () (const pair<string, string>& p) const {
        hash<string> hasher;
        auto h1 = hasher(p.first);
        auto h2 = hasher(p.second);
        return h1 ^ h2; // Below for better heuristics.
    }
};

```

Heuristics to combine hash values.
 $h2 = h2 \oplus (h1 \ll 1)$; $h2 \oplus= h2 + 0x9e3779b9 + (h1 \ll 6) + (h1 \gg 2)$;
 0x9e3779b9 is a prime number.
 See: <https://stackoverflow.com/a/2595226/5607735>

Others

- `swap(T& a, T&b);`
- `reverse(Iterator first, Iterator last);`
- `next_permutation(a.begin(), a.end());`
- `prev_permutation(a.begin(), a.end());`
- `random_shuffle(a.begin(), a.end());`
 Gives the next permutation for a given vector of integers.

6. Libraries

6.1. Input output

File Access

Format:
 FILE *fp;
 FILE *fopen(char *name, char *mode);
 int fclose(FILE *fp);
 fp = fopen(name, mode);
 Allowable modes include:
 "r" read.
 "w" write.
 "a" append.
 "b" binary files: "rb", "wb", "ab".
 NOTE: **Difference between rb and r etc:**
 While using just "r" might work in writing binary files, it might cause trouble due misinterpretation of special characters like LF and CR.
 See: <https://stackoverflow.com/q/2174889/5607735>
Standard IO Can be treated in the same way as file pointer.
 stdin, stdout, stderr

Reading and writing a file

```

size_t fread(const void* ptr, size_t size,
size_t count, FILE* stream);

size_t fwrite(const void* ptr, size_t size,
size_t count, FILE* stream);

```

Character I/O

```

getchar    Takes input from standard input.
            int getchar()
            Equivalent togetc(stdin)
putchar    Output to standard output.
            int putchar(int)
            Equivalent toputc(c), stdout
getc       int getc(FILE *fp), Eg: getc(stdin)
putc       int putc(int c, FILE *fp)
ungetc

```

Line I/O

```

gets       Reads until EOF.
            gets deletes terminal \n
puts       Writes line to stdout.
            puts adds terminal \n
fgets      char *fgets(char *line, int maxline, FILE *fp)
fputs      int *fputs(char *line, FILE *fp)
getline    Eg: getline(cin, s). Where s is a string.

```

NOTE: Never use gets
 gets does not check for buffer overrun, and keeps reading until it encounters new line or EOF. **It has been used to break computer security.**

printf and scanf

Conversion formats for printf and scanf

Character	Argument type; Printed as
d,i	int; decimal number
o	int; unsigned octal number (without leading zero)
x,X	int: unsigned hexadecimal number (without a leading 0x or 0X), using abcdef or ABCDEF.
u	int; unsigned decimal number.
zu	size_t.
c	int; single character.
s	char *; Print string, scan word. NOTE: scanf reads only a word and not the entire string.
f	double; [-]m.ddddd.
e,E	double; [-]m.dddddd±xx, or [-]m.dddddd±xx.
g,G	double; %e or %E if exponent is < -4 or >= precision, else use %f.
p	void *; pointer.
%	no argument; Print %.

Between % and conversion character there may be in order:

- Minus sign: Left justification.
- Number: Minimum field width.
- Period: Separates field width from precision.
- Number: Precision.
 For float: number of digits after decimal.
 For int: minimum number of digits.
 For string: maximum number of characters to be printed.
- h: for short integer. l: for long integer.

Formatting rules

Examples

%*d
 Here precision can be specified dynamically. Eg:
`printf("%*d", 6, foo);`, this is equivalent to
`printf("%6d",foo);`

Integers
 %d
 print as decimal integer.
 %6d
 print as decimal integer, at least 6 characters wide.

Floating point numbers
 %6f
 print as floating point.
 %.2f
 print as floating point, 2 characters after decimal point.
 %6.2f
 print as floating point, at least 6 characters wide and 2 characters after decimal point.

String: example hello, world (12 chars).
 %s: :hello,_world:
 %10s: :hello,_world:
 %.10s: :hello,_wor:
 %-10s: :hello,_world:
 %.15s: :hello,_world:
 %-15s: :hello, world_:_:
 %15 .10s: :_._._hello, wor:
 %-15 .10s: :hello,_wor_._._._:

Printf and variants

- `int printf(char *format, arg1, arg2, ...);`
Print output to stdout.
- `int sprintf(char *string, char *format arg1, arg2, ...);`
Print output to `string`.
- `int fprintf(FILE *fp, char *format arg1, arg2, ...);`
Print output to file pointed by the file pointer `fp`.

Example:

```
printf("%s\n", string_var);
printf("%s", "hello, world");
printf("square of n is %d\n", n_square);
```

Scanf and variants

- `int scanf(char *format, arg1, arg2, ...);`
Scan input from stdin.
- `int sscanf(char *string, char *format arg1, arg2, ...);`
Scan input from `string`.
- `int fscanf(FILE *fp, char *format arg1, arg2, ...);`
Scan input from the file pointed by the file pointer `fp`.

NOTE1: unlike `printf`, in `scanf` the variable are pointers.

NOTE2: In `scanf %s` reads a word and not the entire string.

Also see: <https://stackoverflow.com/a/1248017/5607735>

Examples:

```
scanf("%d", &n);
sscanf(string, "%d", &n);
fscanf(fp, "%d", &n);
```

6.2. cin and cout

<code>cin</code>	Read input until space, or tab or LF.
<code>cout</code>	Output.

NOTE:

- Use the following to increase speed of I/O.
`ios_base::sync_with_stdio(false);`
`cin.tie(nullptr); cout.tie(nullptr);`
- Use `cin.ignore()` after `cin` and before using `getline`.

6.3. Miscellaneous

```
int fflush(FILE *stream)
```

Causes any buffered but unwritten data to be written. The effect is undefined on an input stream.

```
int fclose(FILE *stream)
```

Flushes any unwritten data for `stream`, discards any unread buffered input.

6.4. String

A `string` is like a vector of characters.

Definition:

```
string s = "Hello";
```

String operations:

<code>a + b</code>	Concatenate.
<code>s.append(t)</code>	append <code>t</code> to <code>s</code> .
<code>s.compare(t)</code>	returns 0 if <code>s == t</code> , returns <code>< 0</code> if <code>s < t</code> , else returns <code>> 0</code> .
<code>s.substr(start, len)</code>	Select a substring.
<code>s.find(str2)</code>	find first occurrence of <code>str2</code> .
<code>s.find(str2, pos)</code>	Find first occurrence of <code>str2</code> by searching from the position <code>pos</code> .

<code>x</code>	Hex.
<code>o</code>	Octal.
<code>t</code>	Binary.
<code>d</code>	Decimal.
<code>u</code>	Unsigned decimal.
<code>i</code>	Instruction.
<code>c</code>	Character.
<code>s</code>	String.

<code>b</code>	byte.
<code>h</code>	Halfword.
<code>w</code>	Word.
<code>g</code>	Giant.

7. Devtools

7.1. G++

Usage:

```
g++ option hello.cpp -o hello
```

```
g++ hello.cpp -o hello
```

```
g++ -O3 hello.cpp -o hello
```

Options:

<code>-o</code>	Output file name.
<code>-I</code>	Include additional directories in the search path. Eg: <code>g++ -I /home/user/libs foo.cpp</code>
<code>-O<level></code>	Optimization level, starts from 0. Eg: <code>-O3</code>
<code>-g</code>	Creates breakpoints for debugging using GDB.
<code>-lm</code>	Missing link. Used to link certain libraries like <code>math.h</code> .
<code>-E</code>	Output preprocessed but uncompiled code.
<code>-S</code>	Compile but do not assemble.
<code>-c</code>	Compile and assembly, but do not link.

7.2. GDB

To run: `gdb hello`

<code>break</code>	Set break point. Eg: <code>break function</code> or <code>break line number</code> .
<code>run</code>	Run the program. The program stops at every break point.
<code>next</code>	Run until next breakpoint.
<code>print</code>	Eg: <code>print i</code> or <code>print &i</code>
<code>sizeof</code>	Eg: <code>print sizeof(i)</code>
<code>&i</code>	Address of <code>i</code> .
<code>*j</code>	Content of memory location <code>j</code> .
<code>ptype</code>	get type of a variable. Eg: <code>ptype(i)</code>
<code>set var</code>	Reassign variable. Eg: <code>set var i = 1</code>
<code>disassemble</code>	Use after break and run. Gives assembly code.

Accessing memory location using x

Usage: `x/nfs`. `nfs` describes the format.

`n` - Number of units to display.

`f` - Number format.

`s` - Size of each unit.